Saurabh Kumar

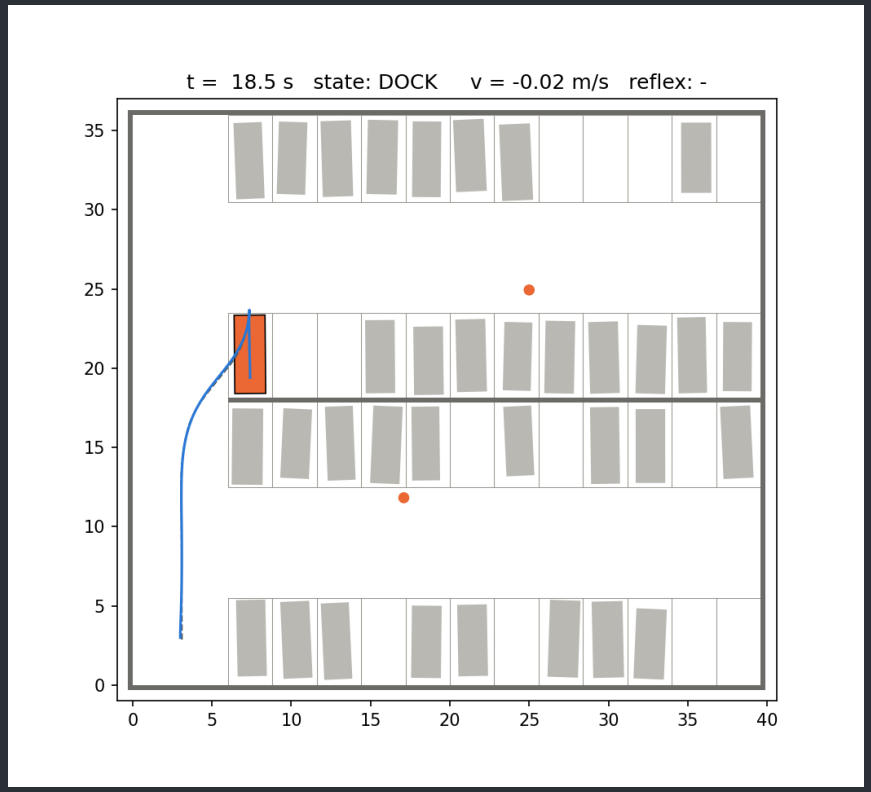
Autonomous Valet Parking Agent

Hybrid A* planning · Pure Pursuit + PID · reflexive safety layer · Webots

Project P13 | Group G14 | Batch B2

Faculty: Course Faculty, CSAI4014, School of Computer Science, UPES

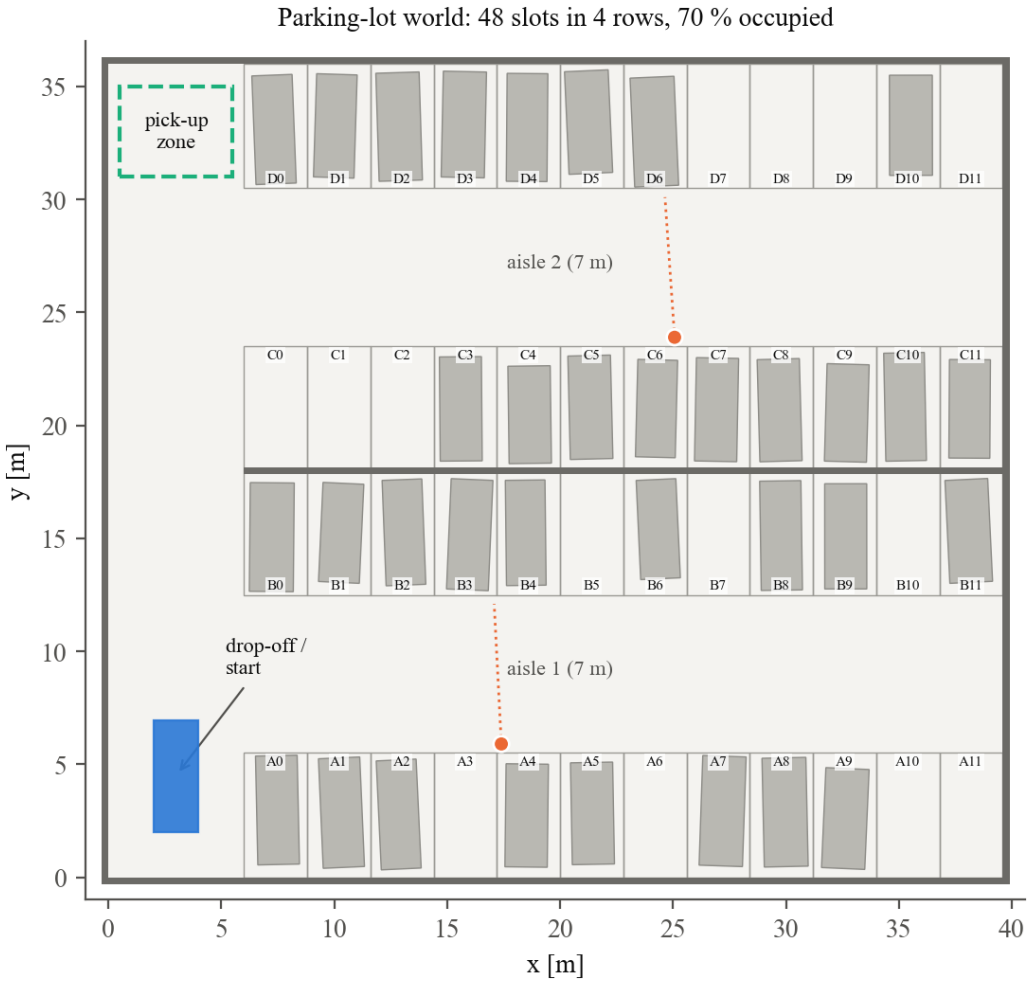
Saurabh Kumar	500124288
Naman Chanana	500119485
Anirvan Ranjan	500119330
Aditya Pratap Singh Sisodia	500121985
Priyal Sharma	500122041



Problem and motivation

Saurabh Kumar

- **Parking:** slow, stressful, and where most low-speed scrapes happen
- **Automated valet parking (AVP):** leave the car at drop-off, it parks itself and comes back to pick-up
- **Industry:** Bosch + Mercedes Level-4 AVP, Stuttgart airport (2022); ISO 23374 standard
- **Hard because:** a car can't turn on the spot (R_min = 4.87 m), must reverse, and must react to people



Our lot: 48 slots, 2 aisles, 50-92 % full, crossing pedestrians

Objectives: target vs achieved

Saurabh Kumar

#	Measurable objective	Target	Achieved
O1	Success rate, 70 % full lot, zero collisions	≥ 95 %	100 % (30/30)
O2	Final parking accuracy	< 10 cm, < 2°	4.2 cm, 1.2°
O3	Planning time in Python	< 5 s	1.65 s mean
O4	Pedestrian contacts (4 crossing people)	0 %	0 %
O5	Methods compared per layer	≥ 2	2 planners, 2 steering laws, 3 reflexes, 2 slot policies

Existing solutions and the gap

Naman Chanana

Approach	Strength	Gap
Grid A* (Hart et al. 1968)	Fast, optimal on grid	Ignores heading: path not drivable
Hybrid A* (Dolgov et al. 2010)	Drivable, obstacle-aware	Slow to hit a narrow slot pose
Pure Pursuit / Stanley	Simple, proven	Stanley weak in reverse
Park-assist rules	Cheap ultrasonic stop	Fixed distance misses side-swipes
Industry AVP (Bosch, ISO 23374)	Certified, Level 4	Closed source, costly infrastructure

Gap we target: fast slot planning + turning-aware safety + choosing the slot for the whole valet cycle

Novelty: what is new, with evidence

Naman Chanana

5.5× fewer

node expansions with our slot "goal funnel" in Hybrid A* (success 80 % → 100 %)

0 % contacts

with the swept-footprint TTC reflex (17 % fixed-distance, 20 % none)

−7.6 %

valet cycle length (drive in + out) with the utility-based slot choice

100 % vs 20 %

success at 0.3 m GPS noise: Kalman filter vs raw GPS

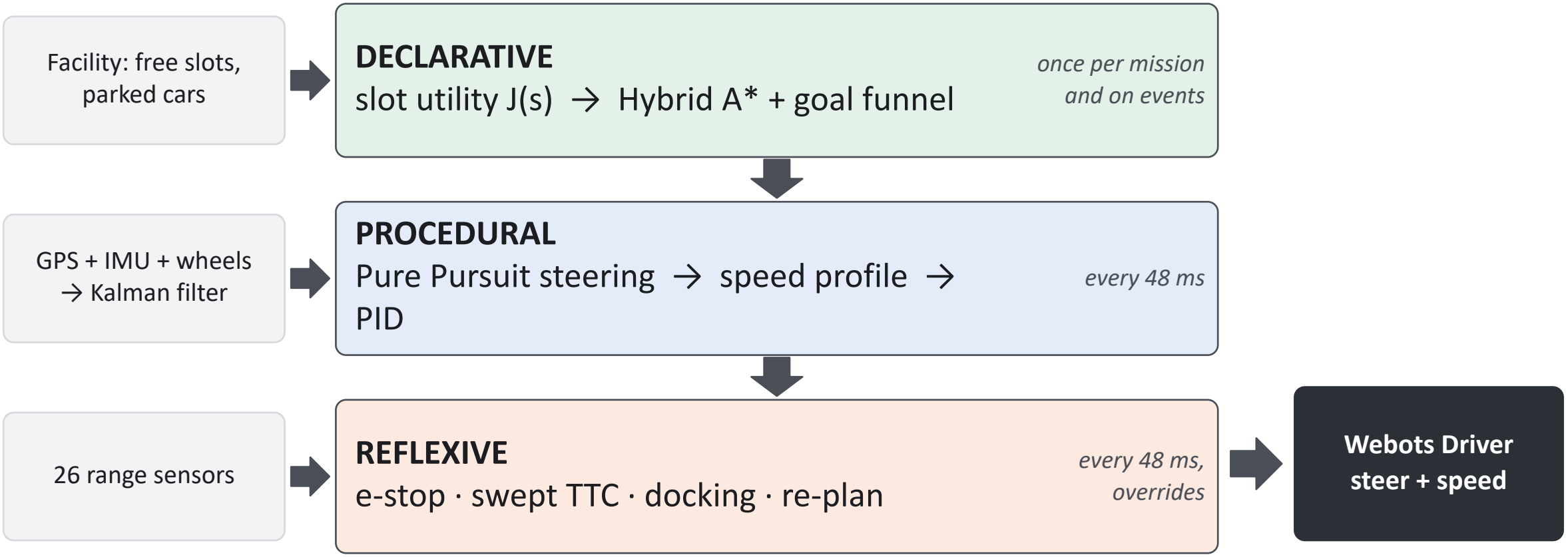
AISR syllabus mapping

Anirvan Ranjan

Syllabus topic	Where we use it
Declarative–procedural–reflexive hierarchy	The three layers of the agent
Intelligent agents	Utility-based slot choice, mission state machine
Path planning, planning to move	Grid A*, Hybrid A* + goal funnel
Moving (ground robots)	Ackermann bicycle model, reversing
Closed-loop control, time/frequency domain	PID speed loop, step response, Bode, phase margin
Perception, uncertain movement	26 range sensors, Kalman filter, Monte Carlo tests

System architecture: three-layer agent

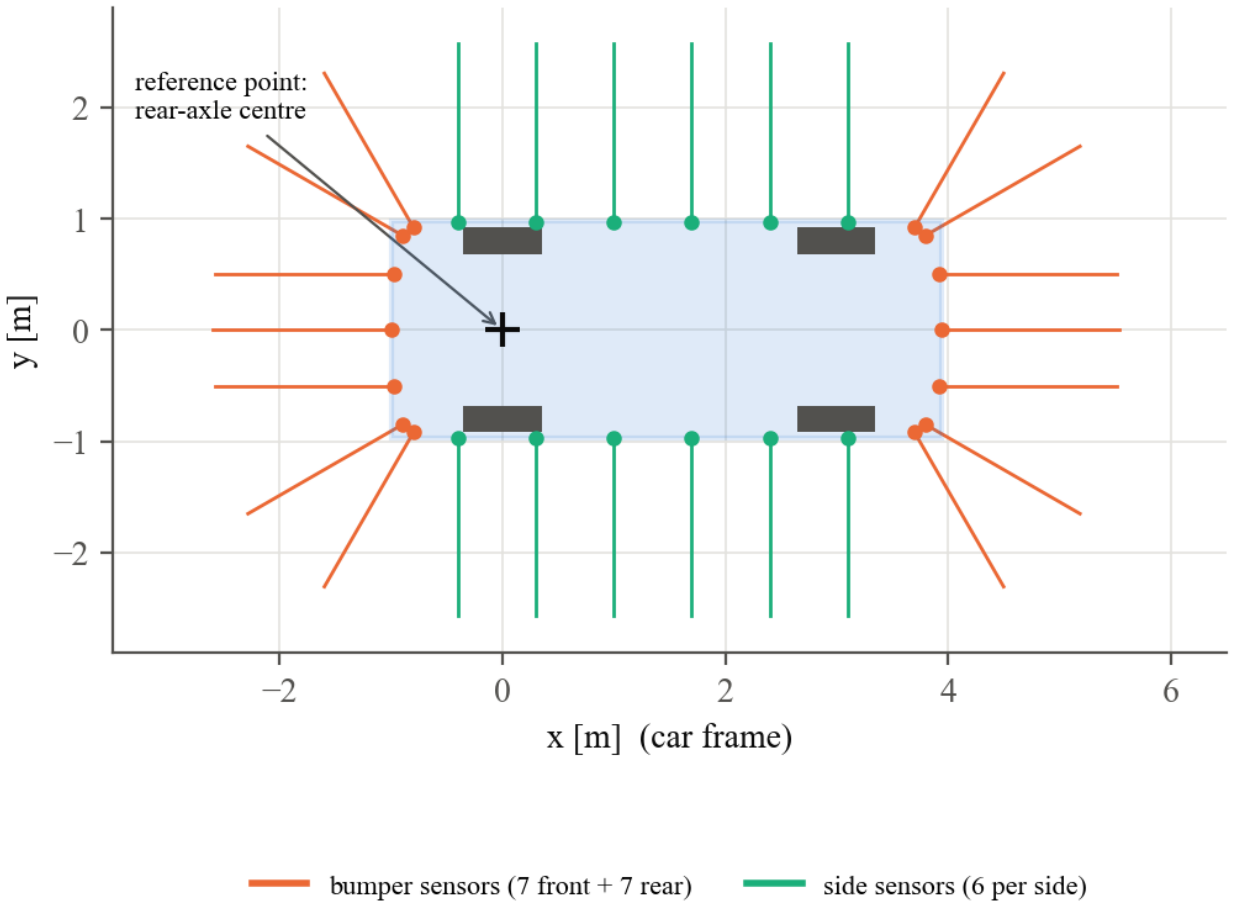
Anirvan Ranjan



Mission FSM: SELECT → PLAN → DRIVE → DOCK → PARKED (re-select if slot taken, re-plan if blocked)

Robot and environment model

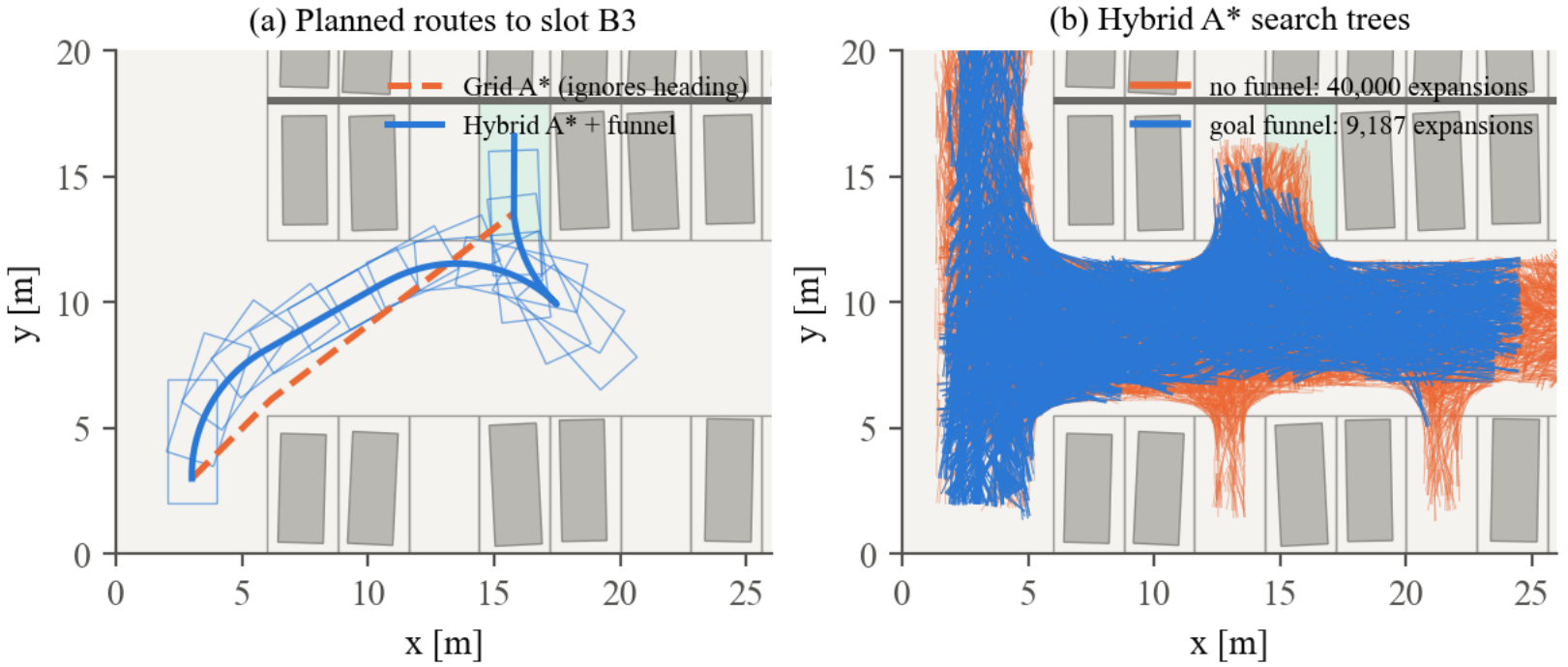
Saurabh Kumar



- **Car:** Webots BMW X5, Ackermann
- **Bicycle model:** $\dot{\theta} = v \cdot \tan\delta / L$, $L = 2.995$ m
- **$R_{min} = L / \tan\delta_{max} = 4.87$ m**
- 26 range sensors, GPS, IMU, wheel speed
- World generated from the same Python lot model (no drift between Webots and twin)

Algorithm 1: Hybrid A* with goal funnel

Naman Chanana

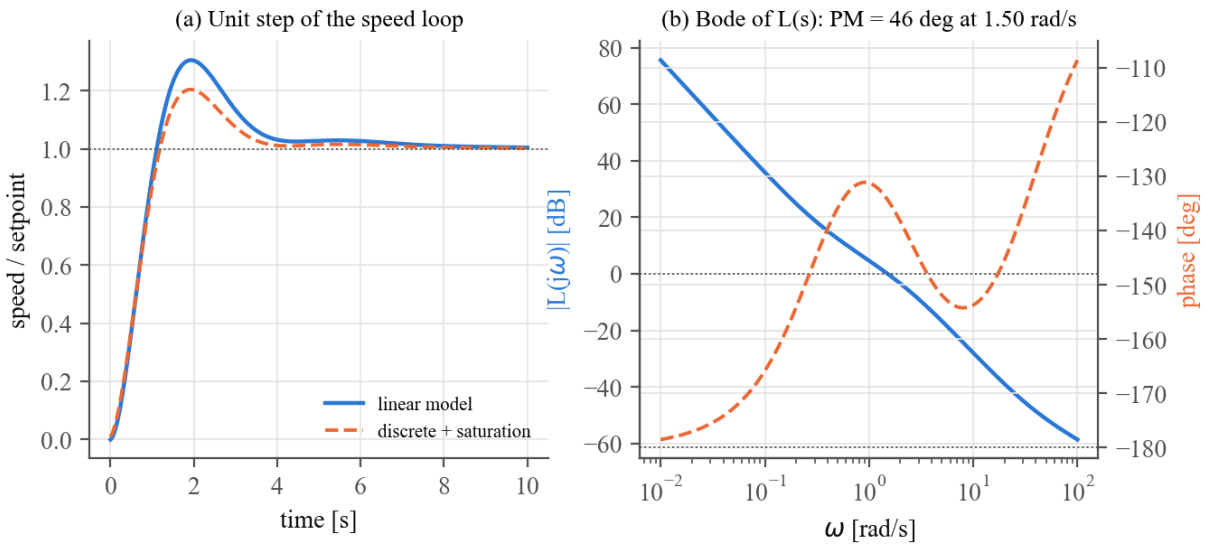


1. Expand Ackermann primitives (5 steers × fwd/rev, 1 m)
2. Cost: length, reverse ×1.6, steering, gear changes
3. Funnel: stop on the slot axis (±0.2 m, ±6°), finish with a straight reverse

Algorithm 2: path tracking and speed control

Aditya P. S. Sisodia

- **Pure Pursuit:** $\kappa = 2 \sin \alpha / L_d$, $\delta = \pm \text{atan}(L \kappa)$ (sign flips in reverse)
- **Look-ahead tuned:** $L_d = \max(1.6, 0.8|v|)$
- **PID speed:** Kp 1.8, Ki 0.6, Kd 0.05, anti-windup
- **Lag-aware braking** removed a 0.35 m stop overshoot
- **Plan with 88 % steering** → reserve for feedback



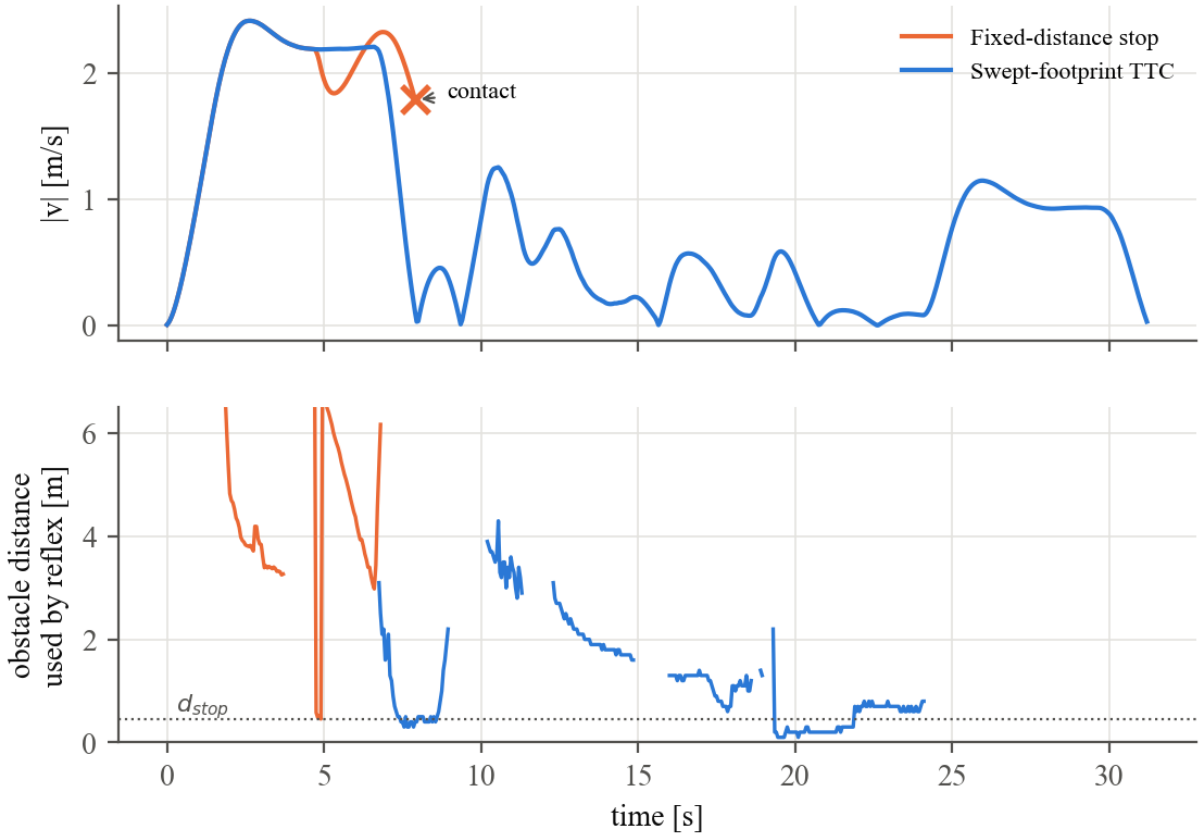
Phase margin 46° · rise 0.85 s · overshoot 20 % (with saturation)

Stanley compared: tracks forward well, but only 57 % of missions park within tolerance (Pure Pursuit 100 %)

Algorithm 3: reflexive layer (swept TTC)

Priyal Sharma

- **R1** e-stop if contact distance < 0.45 m
- **R2** v ≤ (s_c − 0.45) / 2.5 s − v_obstacle
- **R3** docking: centre in slot, rear gap stop
- **R4** blocked 4 s → re-plan around it
- **s_c:** move the car outline along the current steering arc; first sensed point it touches



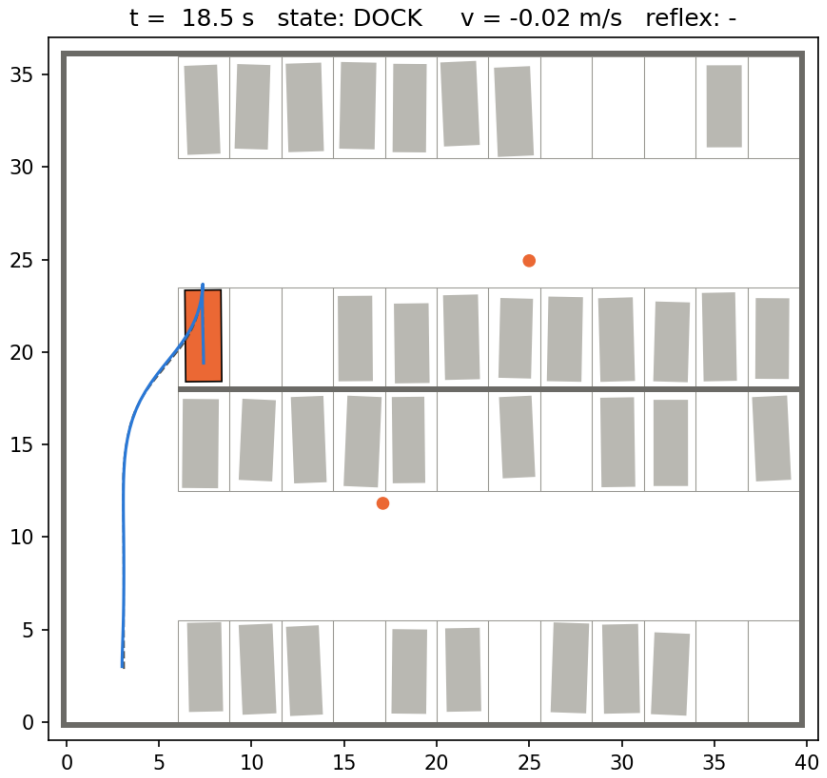
Fixed-distance rule hits the pedestrian (×); swept TTC slows smoothly and parks

Simulation setup and live demo

Saurabh + Aditya

```
python tools/make_world.py
# Webots: open valet_parking.wbt, Run
python sim/run_demo.py
python experiments/run_experiments.py
```

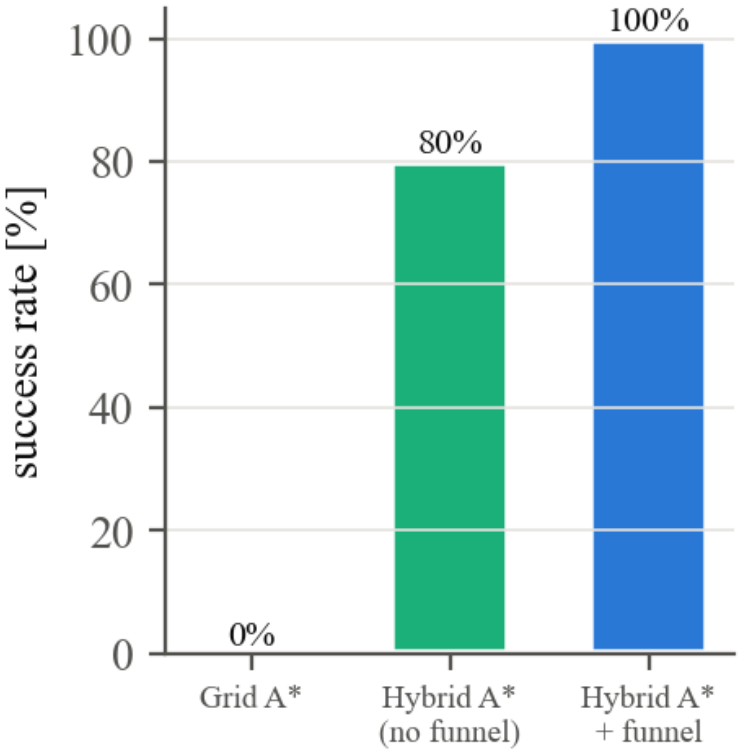
- Webots R2025a, BmwX5 PROTO, Python 3.11 controller
- Headless twin: same agent code, 525 Monte Carlo missions
- Control period 48 ms; 30 random lots per setting

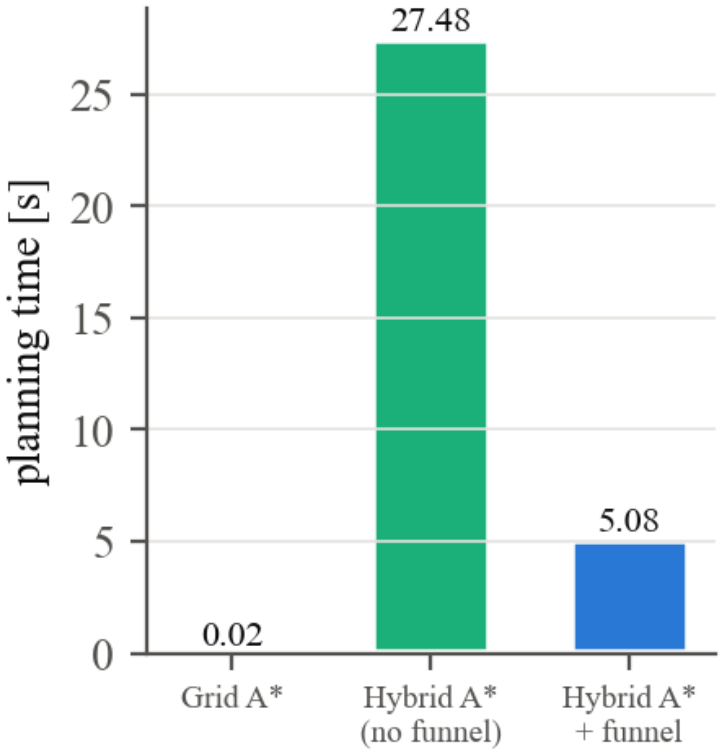


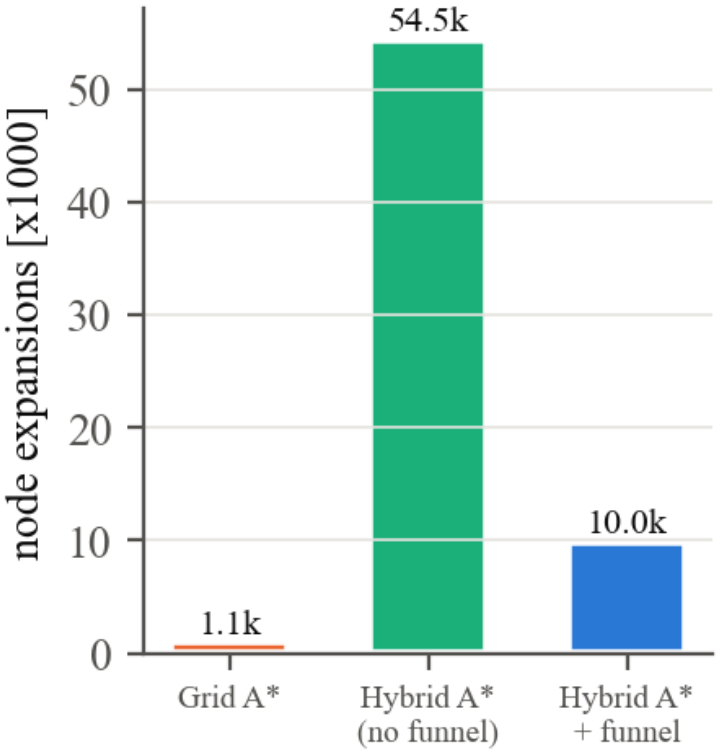
▶ Switch to Webots now (≈ 6 min live demo)

Results 1: planning and tracking

Aditya P. S. Sisodia







0 %

grid A* success (cannot be driven)

100 %

Hybrid A* + funnel success

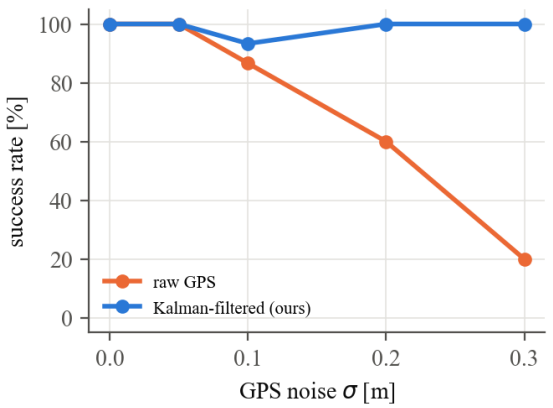
4.2 cm · 1.2°

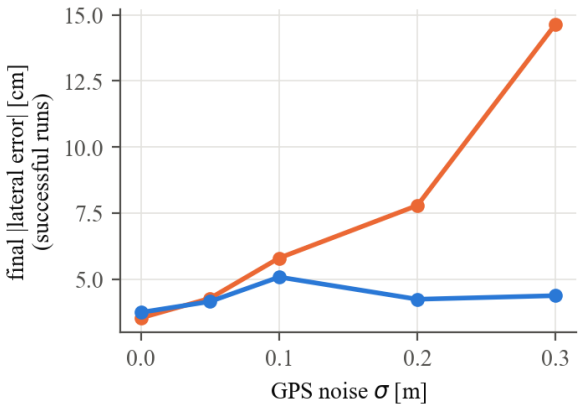
final lateral / heading error

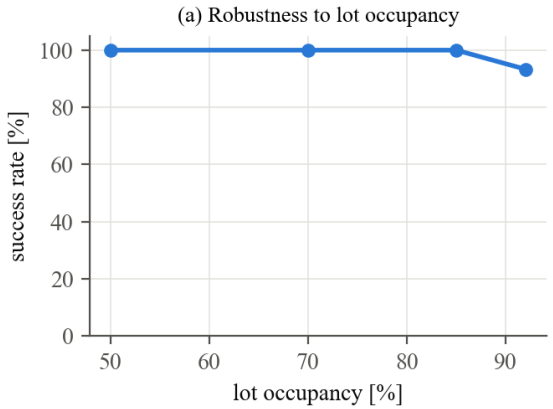
Results 2: safety, robustness, comparison

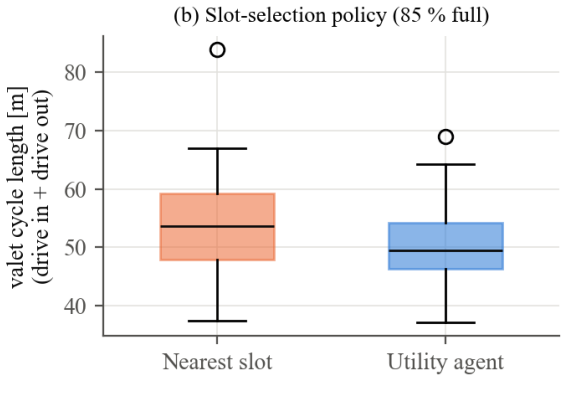
Anirvan Ranjan

Experiment	Baseline	Ours
E3 pedestrian contacts	17 % fixed-distance / 20 % none	0 % (swept TTC)
E4 valet cycle length	54.9 m (nearest slot)	50.7 m (utility agent)
E5 success at 0.3 m GPS noise	20 % (raw GPS)	100 % (Kalman filter)
E6 / E7 92 % full / slot taken	–	93 % / 100 %









Team contribution (traceable to GitHub)

Priyal Sharma

Member	Module and files
Saurabh Kumar (lead)	Lot model, costmap, Webots world generator and controller
Naman Chanana	Grid A*, Hybrid A* + goal funnel, path, unit tests
Anirvan Ranjan	Agent + state machine, utility slot selection, Kalman filter
Aditya P. S. Sisodia	PID, Pure Pursuit, Stanley, path tracker, tuning
Priyal Sharma	Reflex layer (swept TTC), simulator twin, experiments

Report written equally: about 5 pages each, plus own contribution row and reflection

Conclusion and industry use

All members

Findings

- Kinematic planning is essential
- Funnel: 5.5× faster search
- Swept TTC: 0 % contacts
- Utility + KF: robust agent

Industry use

- Infrastructure-supported AVP (ISO 23374)
- Factory yards and ports
- Robotaxi parking and charging

Future scope

- Real car: lidar or ultrasonic sensors, C++
- Parallel and angled slots
- Pedestrian prediction
- Many valet cars sharing a lot

All members

Thank you

Questions?

GitHub: github.com/<team-lead>/g14-valet-parking
Video: youtu.be/<video-id> (unlisted)
Contact: team e-mails in the repository README

Saurabh Kumar · Naman Chanana · Anirvan Ranjan · Aditya Pratap Singh Sisodia · Priyal Sharma